
Deliverable Contracts for Long-Horizon Agents: Gating Agent Runs with Verify–Repair Loops and Tamper-Evident Audits

Zhen Li

zhen@repl.it

github.com/zhenbuilder/charter-gate

Abstract

Long-horizon agent tasks rarely fail loudly. They fail by *declaring victory*: the agent reports completion while deliverables are missing, malformed, or silently dropped. We propose governing agent work the way continuous integration governs human work — by declaring the deliverables themselves as a machine-checkable *contract*, gating every run through a bounded verify–repair loop, and recording each attempt in a hash-chained audit log. We implement this as **CHARTER**, a single-file, dependency-free CLI that wraps *any* agent invocable as a shell command. To anchor the mechanism on a public benchmark, we run a paired A/B on all 164 HumanEval problems with a small model (claude-haiku-4-5): the baseline arm submits the model’s first answer; the gated arm adds a contract requiring the solution to parse, define the entry point, and pass model-authored smoke tests derived *only* from the public docstring, with at most two repair rounds. Hidden tests are never shown. Scored with the official HumanEval harness, accuracy is at ceiling and the gate’s effect on pass@1 is small and not statistically significant ($97.6\% \rightarrow 98.2\%$, +0.6pp, exact McNemar $p = 1.0000$); we report this null result as such. What the run does establish is the gate’s governance behaviour at low cost: it caught first-attempt contract breaches on 17 + 1 of 164 tasks, auto-repaired 17 of them, surfaced the remaining 1 as an explicit nonzero-exit failure instead of a silent claim of completion, with 0 regressions observed, at $1.12\times$ the model calls of the baseline. We release the tool, the full raw logs, and the scoring pipeline; every number in this paper is generated directly from the committed artifacts.

1 Introduction

Agents that work for minutes to hours produce *deliverables*: files, patches, reports, passing test suites. The dominant failure mode in practice is not a crash but an unverified claim of completion — the agent “declares victory” on deliverables that do not exist, do not pass, or quietly omit part of the task. Current mitigations are either *post hoc* (offline evals, human review) or *framework-bound* (bespoke verifier hooks inside a particular agent stack).

Software engineering solved the same problem for humans decades ago: continuous integration. Nobody merges on the author’s word; a declarative pipeline checks the deliverables and blocks on failure. We port this pattern to agent runs with three components:

1. **Contract.** A `charter.toml` file declares deliverables (paths and globs, minimum sizes and counts, required/forbidden regular expressions, freshness) and arbitrary executable acceptance checks (any command; exit 0 means pass).

2. **Gate.** `charter run - <agent command>` executes the agent, verifies the contract, and on breach re-invokes the agent with the structured violation report injected (via an environment variable pointing at the report), up to a bounded number of repair rounds.
3. **Audit.** Every attempt appends a record to `.charter/audit.jsonl` containing the verdict and the SHA-256 of every deliverable, hash-chained to the previous record so after-the-fact tampering is detectable.

The design is deliberately minimal: a single `stdlib`-only Python file, agnostic to the agent (anything that is a shell command), installable by a stranger in seconds. Our claim is not that contracts make a weak model strong; it is that an *external, declarative* definition of “done” — checked, retried, and logged outside the agent’s own control — converts silent victory declarations into either repaired deliverables or explicit, auditable breaches, and yields an evidence trail that an agent’s self-report cannot provide. We evaluate both whether the gate is *safe* (does it harm benchmark accuracy?) and what its governance loop actually does, reporting the accuracy comparison honestly as a null result at ceiling.

Contributions. (i) A problem framing: deliverable-contract governance for long-horizon agent runs, with contract, gate, and audit as separable components. (ii) CHARTER, an open-source, dependency-free reference implementation. (iii) A paired A/B on a public benchmark (HumanEval, official scorer, $n = 164$ per arm) with a repair-loop ablation, withholding hidden tests from the model, with all raw logs and scoring code released — including an honest null result on accuracy and a decomposition of what the gate’s repairs actually changed.

2 Related Work

Self-correction and self-refinement. Prompting an LLM to critique and revise its own output (e.g., Self-Refine [2], Reflexion [3]) improves some tasks, but the critic shares the generator’s blind spots, and unguided self-correction can degrade performance [4]. Our gate differs in that the retry signal is an *externally executed* contract verdict (parse checks, executed tests), not the model’s opinion of itself.

Execution-guided code generation. Using executable tests at generation time is well established: CodeT generates tests to select samples [5], LEVER verifies with execution [6], and self-debugging feeds execution errors back to the model [7]. Our HumanEval arm B is an instance of this family; the contribution is not the specific signal but its packaging as a general, declarative, agent-agnostic contract that also covers non-code deliverables and produces an audit chain.

Agent evaluation harnesses. SWE-bench [8], GAIA [9], and HumanEval [1] score agent or model outputs offline against hidden references. Contracts are complementary: they run *inline*, use only information the operator legitimately has (the spec), and gate the run rather than grade it afterwards.

Policy-as-code. Declarative gates over artifacts are standard in infrastructure (OPA, branch protection, pre-commit). CHARTER transplants that pattern to agent deliverables and adds the bounded repair loop and hash-chained attempt log.

3 The CHARTER system

3.1 Contract language

A contract is a TOML file with two block types. `[[deliverable]]` declares a path or glob with optional `min_count`, `min_bytes`, `must_match` / `must_not_match` regular expressions, and `max_age_minutes` (freshness). `[[check]]` declares an arbitrary command with a timeout; exit status 0 is a pass. `charter verify` evaluates all blocks, prints a human-readable and (optionally) JSON report, and exits nonzero on any breach — making it directly usable as a CI step.

3.2 Verify-repair gate

`charter run -max-repairs  $k$  - <agent cmd>` executes the agent in the workspace, then verifies. On breach, it writes the violation report to a file, sets `CHARTER_VIOLATIONS` to its path, and

re-invokes the same agent command; a contract-aware driver includes the report (and its previous outputs) in the next prompt. The loop is bounded by k , so a contract that the agent cannot satisfy terminates with a nonzero exit rather than an infinite loop.

3.3 Tamper-evident audit

Each attempt appends `{attempt, agent_exit, ok, deliverable_sha256s, prev}` to `.charter/audit.jsonl`, where `prev` is the SHA-256 of the previous record; `charter log` re-verifies the chain. The operator can therefore prove which bytes the agent delivered on which attempt, and detect post-hoc modification of the log itself.

4 Experimental setup

Benchmark and model. We use all 164 problems of HumanEval [1] and score with the official open-source harness (vendored unmodified; the one documented line enabling execution is uncommitted). The subject model is `claude-haiku-4-5` invoked through the `claude` CLI; we chose a small model because frontier models are near ceiling on HumanEval, which would make any gate effect unmeasurable. HumanEval is not itself long-horizon; we use it because its unit of evaluation *is* a deliverable with an acceptance test, the property the gate targets, and because it admits official, local, reproducible scoring within our compute budget (Section 7 discusses external validity).

Conditions (paired, per task). **A (baseline):** one model call; the produced `solution.py` is submitted as-is. **B (gated):** the same model and task prompt, but the model must produce both `solution.py` and `smoke_test.py`, a self-contained test derived *only* from the public docstring. A per-task contract requires: the solution file exists with the entry-point definition; it parses (`ast.parse`); the smoke test exists, imports the solution, and passes when executed. The gate runs with `-max-repairs 2`, so at most three model calls per task. *The hidden HumanEval tests are never shown to the model or the contract in either arm.* The extra information available to arm B is exactly what an operator has: the spec and the ability to execute what the agent wrote.

Scoring and statistics. Official harness `pass@1` per arm; Wilson 95% confidence intervals; and, because the design is paired, an exact McNemar test (binomial test on discordant pairs). We additionally score **B₀**, arm B’s first-attempt solutions re-extracted from the committed raw model outputs, which ablates the repair loop and separates the effect of the arm-B prompt (write tests alongside the solution) from the effect of `verify-repair`. All raw model outputs (`call_n.txt`), per-task metadata, charter audit logs, official per-task verdicts, and the analysis script are committed; `experiments/analyze.py` regenerates every number in this section, including this paper’s macro file.

5 Results

Table 1: Paired A/B on HumanEval ($n = 164$ per arm), official scorer, `claude-haiku-4-5`. Arm B adds the `CHARTER` gate (docstring-derived smoke tests, ≤ 2 repairs); hidden tests withheld in both arms. **B₀** scores arm B’s *first-attempt* solutions (extracted from the committed raw outputs), i.e. arm B with the repair loop ablated.

Arm	pass@1	95% CI	passed/ n	mean model calls
A: single shot	97.6%	[93.9, 99.0]	160/164	1.00
B ₀ : B without repairs	98.2%	—	161/164	1.00
B: <code>CHARTER</code> gate	98.2%	[94.8, 99.4]	161/164	1.12

Accuracy: a null result at ceiling. The baseline already passes 160 of 164 hidden test suites (97.6%), leaving headroom of only four tasks. The gated arm passes 161 (+0.6pp). The paired decomposition shows 1 task fixed and 0 broken (exact McNemar $p = 1.0000$): *the accuracy difference is not statistically significant*, and we report it as a null result. The **B₀** ablation localises the single fixed task (HumanEval/92) to the arm-B *prompt* (writing acceptance tests alongside the

solution), not the repair loop: repairs changed 0 hidden-test outcomes (0 regressions). A gate cannot add correctness that a near-ceiling model does not lack; measuring an accuracy effect would require a harder benchmark or weaker model, which we flag as future work rather than re-rolling the dice here.

Governance behaviour: what the gate actually did. Charter’s audit chains decompose arm B: 146 tasks satisfied the contract on the first attempt; 17 breached it and were auto-repaired within two rounds; 1 ended with an unresolved breach, which the gate surfaced as a nonzero exit — the operator-visible alternative to a silent victory declaration. Among the 17 repaired tasks, 15 were *false alarms* (the first-attempt solution already passed the hidden tests; the model’s own smoke test was wrong, and the repair round fixed the test), while 2 were genuine solution bugs that the bounded repair loop did not fix against hidden tests. The unresolved task’s solution in fact passes the hidden tests (1/1): the gate’s verdicts are exactly as good as the contract, no better. In this run we observed 0 regressions relative to baseline; with a single run and one discordant pair this is evidence of absence of harm only at the run level, not a general safety guarantee.

Cost. The gate cost $1.12\times$ model calls (184 total vs. 164) and raised mean per-task model latency from 11.6s to 19.6s — the price of executing smoke tests and up to two repairs on the 17+1 breaching tasks.

Honest reading. On this benchmark, with this model, the verify–repair gate bought no significant accuracy and mostly adjudicated the model’s own buggy self-tests. What it provably delivered — visible in the committed audit chains — is the governance contract: every task ended either contract-satisfied or with an explicit, machine-readable breach, with zero regressions and a 12% call overhead. We consider publishing the null accuracy result alongside the mechanism more useful to practitioners than a benchmark cherry-picked to flatter the gate.

6 Dogfooding: governing this project’s own deliverables

The repository’s root `charter.toml` declares this project’s five contracted deliverables (deployable CLI, this paper compiled from auto-generated numbers, the benchmark’s raw logs and official scores, a marketing plan, and a static landing page) with size, pattern, and count constraints plus executable checks — including one that re-runs the analysis script. `charter verify` passes on the released revision; the audit chain for the benchmark runs is included. This is anecdotal rather than experimental evidence, but it demonstrates the intended long-horizon use: the contract was written before the deliverables existed and gated the run that produced them.

7 Limitations

Ceiling effect / null accuracy result. `claude-haiku-4-5` passes 97.6% of HumanEval single-shot, leaving four tasks of headroom; no gate could show a meaningful accuracy effect in this regime, and ours did not. The accuracy claim of this paper is therefore only “0 regressions were observed in this run” — which does not establish harmlessness in general, given one run and one discordant pair; any positive accuracy claim needs a harder benchmark (SWE-bench Lite with a full scaffold is the natural next step) or a weaker model. **Self-verification quality.** The gate is only as good as the contract. Here 15 of 17 repairs fixed the model’s own buggy smoke test rather than the solution, and the one unresolved breach was itself a false alarm; docstring examples also partially overlap hidden tests. Operators who write their own acceptance checks (the intended deployment) sidestep the self-authored-test failure mode but inherit contract-authoring cost. **External validity.** HumanEval is short-horizon and single-file; it anchors the gate’s mechanics on a public, officially-scored benchmark, not the full long-horizon claim. **Single model, single benchmark, $n = 1$ run per arm; no variance estimate across reruns.** **Scope.** Contracts check artifacts, not intent; a malicious agent could satisfy the letter of a weak contract. The audit chain is *tamper-evident*, not *tamper-proof*.

8 Conclusion

Treating “done” as a contract — declared outside the agent, verified by execution, retried under a bound, and logged in a tamper-evident chain — is a cheap, agent-agnostic way to govern the deliverables of long-horizon agent runs. On a public benchmark at ceiling, the gate changed accuracy by a statistically insignificant amount (reported here as a null result) while doing what governance tooling must do: it converted every run into either a contract-satisfied deliverable or an explicit, auditable breach, with zero regressions observed in this run, at 12% extra model calls. The tool, the full raw logs, and the analysis that generated every number above are public: <https://github.com/zhenzhenbuilder/charter-gate>.

References

- [1] M. Chen et al. Evaluating large language models trained on code. *arXiv:2107.03374*, 2021.
- [2] A. Madaan et al. Self-Refine: Iterative refinement with self-feedback. In *NeurIPS*, 2023.
- [3] N. Shinn et al. Reflexion: Language agents with verbal reinforcement learning. In *NeurIPS*, 2023.
- [4] J. Huang et al. Large language models cannot self-correct reasoning yet. In *ICLR*, 2024.
- [5] B. Chen et al. CodeT: Code generation with generated tests. In *ICLR*, 2023.
- [6] A. Ni et al. LEVER: Learning to verify language-to-code generation with execution. In *ICML*, 2023.
- [7] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *ICLR*, 2024.
- [8] C. Jimenez et al. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.
- [9] G. Mialon et al. GAIA: A benchmark for general AI assistants. In *ICLR*, 2024.